



KUBERNETES

Certified Kubernetes Administrator

CKA 2024 — Guía Completa de Estudio

Guía Definitiva de Administración
De Principiante a Administrador Certificado

CONTENIDO INCLUIDO

 Arquitectura & Instalación

 Storage & Persistence

 Networking & Services

 Workloads & Scheduling

Security & RBAC

 Troubleshooting & Debug

+ 200 Comandos Prácticos — Ejercicios CKA — Laboratorios

✓ Actualizado 2024 — ✓ Ejemplos Reales — ✓ Best Practices

Enero 2026 — Versión 1.30+

Índice

1. Introducción a Kubernetes	4
1.1. ¿Qué es Kubernetes?	4
1.1.1. Características Principales	4
1.2. Arquitectura de Kubernetes	4
1.2.1. Control Plane (Plano de Control)	4
1.2.2. Worker Nodes (Nodos de Trabajo)	5
2. Cluster Architecture, Installation & Configuration	6
2.1. Role Based Access Control (RBAC)	6
2.1.1. Componentes de RBAC	6
2.1.2. Ejemplo Práctico de RBAC	6
2.1.3. Comandos Útiles para RBAC	6
2.2. Instalación de Kubernetes con kubeadm	7
2.2.1. Preparación del Sistema	7
3.5. Container Network Interface (CNI)	7
3.5.1. Plugins CNI Populares	7
3.5.2. Instalación de Calico	7
3.5.3. Network Policies con Calico	7
4. Workloads & Scheduling	8
4.1. Pods	8
4.1.1. Anatomía de un Pod	8
4.1.2. Ciclo de Vida de un Pod	8
4.1.3. Comandos Útiles para Pods	8
4.2. Deployments	8
4.2.1. Ejemplo de Deployment	9
4.2.2. Estrategias de Deployment	9
4.3. ConfigMaps y Secrets	9
4.3.1. ConfigMaps	9
4.3.2. Usar ConfigMaps en Pods	10
4.3.3. Secrets	10
4.3.4. Tipos de Secrets	10
4.4. Scaling de Aplicaciones	10
4.4.1. Scaling Manual	10
4.4.2. Horizontal Pod Autoscaler (HPA)	10
4.4.3. Vertical Pod Autoscaler (VPA)	11
4.5. Pod Scheduling	11
4.5.1. NodeSelector	11
4.5.2. Node Affinity	11
4.5.3. Pod Affinity y Anti-Affinity	12
4.5.4. Taints y Tolerations	12
4.5.5. Resource Limits y Requests	12
4.5.6. Quality of Service (QoS)	12
5. Security	14
5.1. Autenticación en Kubernetes	14
5.1.1. Métodos de Autenticación	14
5.1.2. Creación de Certificados de Usuario	14
5.1.3. Configuración de kubeconfig	15

5.2.	Service Accounts	15
5.2.1.	Gestión de Service Accounts	15
5.2.2.	Uso de Service Accounts en Pods	15
5.3.	Security Contexts	15
5.3.1.	Security Context a nivel de Pod	15
5.3.2.	Políticas de Seguridad Recomendadas	16
5.4.	Network Policies	16
5.4.1.	Componentes de Network Policy	16
5.4.2.	Ejemplo de Network Policy Completa	16
5.4.3.	Ejemplos Prácticos de Network Policies	16
5.5.	Pod Security Standards	16
5.5.1.	Niveles de Pod Security	17
5.5.2.	Implementación con Admission Controller	17
6.	Troubleshooting & Monitoring	18
6.1.	Troubleshooting de Clusters	18
6.1.1.	Verificación del Estado del Cluster	18
6.1.2.	Troubleshooting de Nodos	18
6.1.3.	Troubleshooting de Pods	18
6.1.4.	Problemas Comunes y Soluciones	18
6.2.	Troubleshooting de Redes	19
6.2.1.	Conectividad entre Pods	19
6.2.2.	Troubleshooting de Services	19
6.3.	Monitoring con Métricas	19
6.3.1.	Metrics Server	19
6.3.2.	Comandos de Monitoreo Útiles	20
6.3.3.	Troubleshooting de Performance	20
6.4.	Backup y Disaster Recovery	20
6.4.1.	Estrategias de Backup	20
6.4.2.	Script de Backup Automatizado	20
7.	Comandos Esenciales y Cheat Sheet	21
7.1.	Comandos Fundamentales	21
7.1.1.	Gestión Básica	21
7.1.2.	Pods y Deployments	21
7.1.3.	Services y Networking	21
7.2.	Comandos de Troubleshooting	21
7.3.	Comandos de Administración	22
8.	Ejemplos Prácticos del Examen CKA	23
8.1.	Escenarios de Cluster Architecture	23
8.1.1.	Escenario 1: Backup y Restore de etcd	23
8.1.2.	Escenario 2: Actualización de Cluster	23
8.2.	Escenarios de Networking	23
8.2.1.	Escenario 3: Configurar Ingress	23
8.3.	Escenarios de Security	24
8.3.1.	Escenario 4: Configurar RBAC	24
8.4.	Escenarios de Troubleshooting	24
8.4.1.	Escenario 5: Pod no puede iniciar	24

9. Apéndices	25
9.1. Recursos Útiles	25
9.1.1. Documentación Oficial	25
9.1.2. Herramientas de Práctica	25
9.2. Configuración de Laboratorio	25
9.2.1. Ambiente con Vagrant	25
9.2.2. Script de Instalación Automatizada	25
9.3. Glosario de Términos	25
10. Conclusión	27
10.1. Puntos Clave para el Examen	27
10.2. Plan de Estudio Recomendado	27
10.3. Comandos para Memorizar	27
10.4. Última Recomendación	28

1 Introducción a Kubernetes

Información

Kubernetes es una plataforma de orquestación de contenedores de código abierto que automatiza el despliegue, escalado y gestión de aplicaciones en contenedores.

1.1 ¿Qué es Kubernetes?

Kubernetes (también conocido como K8s) es un sistema de orquestación de contenedores que fue desarrollado originalmente por Google y ahora es mantenido por la Cloud Native Computing Foundation (CNCF). Su nombre proviene del griego y significa "timonel" o "piloto".

1.1.1 Características Principales

-  **Orquestación de Contenedores:** Gestiona automáticamente la vida útil de los contenedores
-  **Escalado Automático:** Escala aplicaciones hacia arriba o abajo según la demanda
-  **Auto-sanación:** Reinicia contenedores fallidos, reemplaza nodos, etc.
-  **Load Balancing:** Distribuye el tráfico de red automáticamente
-  **Gestión de Secrets:** Manejo seguro de información sensible

1.2 Arquitectura de Kubernetes

Kubernetes sigue una arquitectura maestro-esclavo (master-worker) con los siguientes componentes principales:

1.2.1 Control Plane (Plano de Control)

Tip Profesional

El Control Plane es el cerebro de Kubernetes, donde se toman todas las decisiones de gestión del cluster.

1. API Server (kube-apiserver)

- Punto de entrada principal para todas las operaciones REST
- Valida y configura datos para objetos API
- Sirve como frontend para el estado compartido del cluster

2. etcd

- Base de datos distribuida y consistente
- Almacena toda la configuración del cluster

- Utiliza el algoritmo de consenso Raft

3. **Controller Manager (kube-controller-manager)**

- Ejecuta los controladores del cluster
- Node Controller, Replication Controller, Endpoints Controller
- Service Account & Token Controllers

4. **Scheduler (kube-scheduler)**

- Asigna pods a nodos específicos
- Considera recursos, políticas y restricciones
- Optimiza la distribución de cargas de trabajo

1.2.2 Worker Nodes (Nodos de Trabajo)

1. **Kubelet**

- Agente que se ejecuta en cada nodo
- Se comunica con el API Server
- Gestiona pods y contenedores en el nodo

2. **Kube-proxy**

- Componente de red que mantiene reglas de red
- Permite comunicación de red a los pods
- Implementa parte del concepto de Service

3. **Container Runtime**

- Docker, containerd, CRI-O
- Responsable de ejecutar contenedores
- Gestiona imágenes de contenedores

2 Cluster Architecture, Installation & Configuration

Importante

Esta sección cubre el 25 % del examen CKA y es fundamental para entender cómo funciona Kubernetes internamente.

2.1 Role Based Access Control (RBAC)

RBAC es un método para regular el acceso a recursos informáticos basado en los roles de usuarios individuales dentro de una organización.

2.1.1 Componentes de RBAC

1. Role y ClusterRole

- **Role**: Define permisos dentro de un namespace específico
- **ClusterRole**: Define permisos a nivel de cluster

2. RoleBinding y ClusterRoleBinding

- **RoleBinding**: Vincula un Role a usuarios/grupos en un namespace
- **ClusterRoleBinding**: Vincula un ClusterRole a nivel de cluster

3. Subjects

- Users (usuarios)
- Groups (grupos)
- ServiceAccounts (cuentas de servicio)

2.1.2 Ejemplo Práctico de RBAC

```
# Crear un Role para leer pods apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: namespace: default name: pod-reader rules: - apiGroups: [] resources: ["pods"] verbs: ["get", "watch", "list"]
```

```
# Crear un RoleBinding apiVersion: rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name: read-pods namespace: default subjects: - kind: User name: jane apiGroup: rbac.authorization.k8s.io roleRef: kind: Role name: pod-reader apiGroup: rbac.authorization.k8s.io
```

2.1.3 Comandos Útiles para RBAC

```
# Verificar permisos de un usuario kubectl auth can-i get pods --as=jane
# Verificar todos los permisos kubectl auth can-i --list --as=jane
# Crear ServiceAccount kubectl create serviceaccount my-service-account
# Obtener token de ServiceAccount kubectl create token my-service-account
```

2.2 Instalación de Kubernetes con kubeadm

kubeadm es una herramienta oficial para crear clusters de Kubernetes que funciona rápida y fácilmente.

2.2.1 Preparación del Sistema

3.5 Container Network Interface (CNI)

CNI define plugins para configurar interfaces de red en contenedores Linux.

3.5.1 Plugins CNI Populares

Flannel

- Simple y fácil de configurar
- Usa VXLAN por defecto
- Bueno para comenzar

Calico

- Soporta políticas de red
- Usa BGP para routing
- Alta performance

Weave

- Mesh network automático
- Encryption built-in
- Easy troubleshooting

3.5.2 Instalación de Calico

```
# Instalar Calico kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/tigera-operator
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/custom-resources.yaml
# Verificar instalación kubectl get pods -n calico-system
# Ver nodos Calico kubectl get nodes -o wide
```

3.5.3 Network Policies con Calico

```
# Denegar todo el tráfico por defecto apiVersion: networking.k8s.io/v1 kind:
NetworkPolicy metadata: name: default-deny-all namespace: production spec:
podSelector: {} policyTypes: - Ingress - Egress
# Permitir tráfico específico apiVersion: networking.k8s.io/v1 kind:
NetworkPolicy metadata: name: allow-frontend-to-backend spec: podSelector:
matchLabels: role: backend policyTypes: - Ingress ingress: - from: -
podSelector: matchLabels: role: frontend ports: - protocol: TCP port: 8080
```


4 Workloads & Scheduling

Importante

Esta sección constituye el 20% del examen CKA y cubre la gestión de aplicaciones y programación de pods.

4.1 Pods

Los Pods son la unidad básica de despliegue en Kubernetes, conteniendo uno o más contenedores que comparten almacenamiento y red.

4.1.1 Anatomía de un Pod

```
apiVersion: v1 kind: Pod metadata: name: my-pod labels: app: my-app version:
v1.0 annotations: description: "Mi aplicación principal"spec: containers:
- name: main-container image: nginx:1.20 ports: - containerPort: 80 name:
http env: - name: ENV_VAR value: "production"resources: requests: memory:
"64Mi"cpu: "250m"limits: memory: "128Mi"cpu: "500m"volumeMounts: -name:
config - volumemountPath: /etc/nginx/conf.dlivenessProbe: httpGet: path: /healthport:
80initialDelaySeconds: 30periodSeconds: 10readinessProbe: httpGet: path: /readyport:
80initialDelaySeconds: 5periodSeconds: 5volumes: -name: config - volumeconfigMap:
name: nginx - configrestartPolicy: AlwaysnodeSelector: disktype: ssd
```

4.1.2 Ciclo de Vida de un Pod

Pending: Pod ha sido aceptado pero no todos los contenedores están running

Running: Pod está ejecutándose en un nodo

Succeeded: Todos los contenedores han terminado exitosamente

Failed: Todos los contenedores han terminado, al menos uno falló

Unknown: Estado del pod no puede ser determinado

4.1.3 Comandos Útiles para Pods

```
# Crear un pod imperativo kubectl run nginx --image=nginx:1.20 --port=80
# Obtener pods con más información kubectl get pods -o wide
# Describir un pod kubectl describe pod my-pod
# Ver logs de un pod kubectl logs my-pod -f
# Ejecutar comando en un pod kubectl exec my-pod -it -- /bin/bash
# Reenviar puerto kubectl port-forward pod/my-pod 8080:80
# Eliminar un pod kubectl delete pod my-pod
```

4.2 Deployments

Los Deployments proporcionan actualizaciones declarativas para Pods y ReplicaSets.

4.2.1 Ejemplo de Deployment

```
apiVersion: apps/v1 kind: Deployment metadata: name: nginx-deployment labels:
app: nginx spec: replicas: 3 selector: matchLabels: app: nginx template:
metadata: labels: app: nginx spec: containers: - name: nginx image: nginx:1.20
ports: - containerPort: 80 resources: requests: cpu: 100m memory: 128Mi limits:
cpu: 200m memory: 256Mi strategy: type: RollingUpdate rollingUpdate: maxSurge: 1
maxUnavailable: 1
```

e

```
# Actualizar imagen del deployment kubectl set image deployment/nginx-deployment
nginx=nginx:1.21
# Ver estado del rollout kubectl rollout status deployment/nginx-deployment
# Ver historial de rollouts kubectl rollout history deployment/nginx-deployment
# Hacer rollback a versión anterior kubectl rollout undo
deployment/nginx-deployment
# Hacer rollback a versión específica kubectl rollout undo
deployment/nginx-deployment --to-revision=2
# Pausar un rollout kubectl rollout pause deployment/nginx-deployment
# Resumir un rollout kubectl rollout resume deployment/nginx-deployment
```

4.2.2 Estrategias de Deployment

1. RollingUpdate

- Reemplaza pods gradualmente
- Zero downtime
- Configurable con maxSurge y maxUnavailable

2. Recreate

- Elimina todos los pods antes de crear nuevos
- Causa downtime
- Útil cuando no se pueden ejecutar versiones múltiples

4.3 ConfigMaps y Secrets

ConfigMaps y Secrets permiten separar la configuración del código de la aplicación.

4.3.1 ConfigMaps

Los ConfigMaps almacenan datos de configuración no confidenciales en pares clave-valor.

```
# Crear ConfigMap desde línea de comandos kubectl create configmap app-config
--from-literal=database_host=localhost
--from-literal=database_port=5432
# Crear ConfigMap desde archivo kubectl create configmap nginx-config
--from-file=nginx.conf
# ConfigMap declarativo apiVersion: v1 kind: ConfigMap metadata: name:
game-config data: game.properties: | enemies=aliens lives=3 enemies.cheat=true
ui.properties: | color.good=purple color.bad=yellow allow.textmode=true
```

4.3.2 Usar ConfigMaps en Pods

s

4.3.3 Secrets

Los Secrets almacenan información sensible como passwords, tokens OAuth, y claves SSH.

```
# Crear Secret desde línea de comandos kubectl create secret generic db-secret
--from-literal=username=admin
--from-literal=password=secretpassword
# Crear Secret para Docker registry kubectl create secret docker-registry
regcred
--docker-server=myregistry.com
--docker-username=myuser
--docker-password=mypassword
--docker-email=myemail@example.com
# Secret declarativo apiVersion: v1 kind: Secret metadata: name: mysecret type:
Opaque data: username: YWRtaW4= base64 encoded password: MWYyZDFlMmU2N2Rm
base64 encoded
```

Tip Profesional

Los valores en Secrets deben estar codificados en base64. Usa `echo -n 'mypassword' | base64` para codificar.

4.3.4 Tipos de Secrets

Opaque: Datos de usuario arbitrarios

kubernetes.io/service-account-token: Token de cuenta de servicio

kubernetes.io/dockercfg: Archivo `.dockercfg` serializado

kubernetes.io/tls: Datos para un cliente o servidor TLS

4.4 Scaling de Aplicaciones

4.4.1 Scaling Manual

```
# Escalar deployment kubectl scale deployment nginx-deployment --replicas=5
# Escalar replicaset kubectl scale rs my-replicaset --replicas=3
# Escalar usando archivo kubectl scale --replicas=3 -f deployment.yaml
```

4.4.2 Horizontal Pod Autoscaler (HPA)

El HPA escala automáticamente el número de pods basado en CPU, memoria u otras métricas.

```
# Crear HPA basado en CPU kubectl autoscale deployment nginx-deployment
--cpu-percent=50 --min=1 --max=10
# HPA declarativo apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler
metadata: name: nginx-hpa spec: scaleTargetRef: apiVersion: apps/v1 kind:
Deployment name: nginx-deployment minReplicas: 2 maxReplicas: 10 metrics: -
type: Resource resource: name: cpu target: type: Utilization averageUtilization:
70 - type: Resource resource: name: memory target: type: Utilization
averageUtilization: 80
```

Importante

Para que HPA funcione, necesitas tener el Metrics Server instalado y los pods deben tener resource requests definidos.

4.4.3 Vertical Pod Autoscaler (VPA)

VPA ajusta automáticamente los resource requests y limits de los contenedores.

```
apiVersion: autoscaling.k8s.io/v1 kind: VerticalPodAutoscaler metadata:
name: nginx-vpa spec: targetRef: apiVersion: apps/v1 kind: Deployment
name: nginx-deployment updatePolicy: updateMode: .AutoresourcePolicy:
containerPolicies: - containerName: nginx maxAllowed: cpu: 1 memory: 500Mi
minAllowed: cpu: 100m memory: 50Mi
```

4.5 Pod Scheduling

El scheduling determina en qué nodos se ejecutan los pods.

4.5.1 NodeSelector

La forma más simple de restringir pods a nodos específicos.

```
# Etiquetar un nodo kubectl label nodes worker-1 disktype=ssd
# Pod con nodeSelector apiVersion: v1 kind: Pod metadata: name: nginx spec:
containers: - name: nginx image: nginx nodeSelector: disktype: ssd
```

4.5.2 Node Affinity

Proporciona reglas más expresivas para seleccionar nodos.

```
apiVersion: v1 kind: Pod metadata: name: with-node-affinity spec: affinity:
nodeAffinity: requiredDuringSchedulingIgnoredDuringExecution: nodeSelectorTerms:
- matchExpressions: - key: kubernetes.io/arch operator: In values: - amd64 -
arm64 preferredDuringSchedulingIgnoredDuringExecution: - weight: 1 preference:
matchExpressions: - key: disktype operator: In values: - ssd containers: - name:
with-node-affinity image: nginx
```

4.5.3 Pod Affinity y Anti-Affinity

Permite programar pods basándose en qué otros pods ya están ejecutándose en el nodo.

```
apiVersion: apps/v1 kind: Deployment metadata: name: redis-cache spec: selector:
matchLabels: app: store replicas: 3 template: metadata: labels: app: store spec:
affinity: podAntiAffinity: requiredDuringSchedulingIgnoredDuringExecution:
- labelSelector: matchExpressions: - key: app operator: In
values: - store topologyKey: "kubernetes.io/hostname"podAffinity:
preferredDuringSchedulingIgnoredDuringExecution: - weight: 100 podAffinityTerm:
labelSelector: matchExpressions: - key: security operator: In values: - S1
topologyKey: topology.kubernetes.io/zone containers: - name: web-app image:
nginx:1.16-alpine
```

4.5.4 Taints y Tolerations

Los Taints permiten a un nodo repeler un conjunto de pods, mientras que las Tolerations permiten a los pods programarse en nodos con taints correspondientes.

```
# Aplicar taint a un nodo kubectl taint nodes node1 key1=value1:NoSchedule
# Remover taint kubectl taint nodes node1 key1=value1:NoSchedule-
# Pod con tolerations apiVersion: v1 kind: Pod metadata: name: nginx spec:
containers: - name: nginx image: nginx tolerations: - key: "key1"operator:
.Equal"value: "value1"effect: "NoSchedule key: "key2"operator: .Exists"effect:
"NoExecute"tolerationSeconds: 3600
```

4.5.5 Resource Limits y Requests

Definen cuántos recursos puede usar un contenedor.

```
apiVersion: v1 kind: Pod metadata: name: resource-demo spec: containers:
- name: resource-demo image: nginx resources: requests: memory: "64Mi"cpu:
"250m"ephemeral-storage: "1Gi"limits: memory: "128Mi"cpu: "500m"ephemeral-storage:
"2Gi"
```

Tip Profesional

Requests: Recursos garantizados

Limits: Recursos máximos que puede usar

CPU se especifica en millicores (m) o cores

Memoria en bytes (Mi, Gi, etc.)

4.5.6 Quality of Service (QoS)

Kubernetes asigna una clase QoS a cada pod:

Guaranteed: Requests = Limits para todos los contenedores

Burstable: Al menos un contenedor tiene requests pero no guaranteed

BestEffort: Ningún contenedor tiene requests ni limits

5 Security

Error Común

La seguridad en Kubernetes es crítica. Esta sección cubre el 15 % del examen CKA y incluye autenticación, autorización y políticas de seguridad.

5.1 Autenticación en Kubernetes

Kubernetes soporta múltiples métodos de autenticación para usuarios y service accounts.

5.1.1 Métodos de Autenticación

X.509 Certificates

- Más común para usuarios administradores
- Cliente presenta certificado válido
- CN del certificado = username, O = groups

Service Account Tokens

- Para aplicaciones que corren en el cluster
- JWT tokens automáticamente montados en pods
- Gestionados por el Service Account Admission Controller

Static Token File

- Tokens pre-compartidos en archivo
- No recomendado para producción
- Requiere reinicio del API server para cambios

5.1.2 Creación de Certificados de Usuario

```
# Generar clave privada para usuario openssl genrsa -out john.key 2048
# Crear Certificate Signing Request openssl req -new -key john.key -out john.csr
-subj /CN=john/O=developers"
# Crear CertificateSigningRequest en Kubernetes apiVersion:
certificates.k8s.io/v1 kind: CertificateSigningRequest metadata: name: john
spec: request: (cat john.csr|base64|tr -d '\n') signerName : kubernetes.io/kube-apiserver-client
usages : -clientauth
# Aprobar el CSR kubectl certificate approve john
# Obtener el certificado kubectl get csr john -o jsonpath='{.status.certificate}'
| base64 -d >john.crt
```

5.1.3 Configuración de kubeconfig

```
# Configurar cluster kubectl config set-cluster kubernetes
--server=https://k8s-api:6443
--certificate-authority=ca.crt
# Configurar usuario kubectl config set-credentials john
--client-certificate=john.crt
--client-key=john.key
# Configurar contexto kubectl config set-context john@kubernetes
--cluster=kubernetes
--user=john
--namespace=development
# Usar contexto kubectl config use-context john@kubernetes
```

5.2 Service Accounts

Las Service Accounts proporcionan identidad para procesos que corren en pods.

5.2.1 Gestión de Service Accounts

```
# Crear Service Account kubectl create serviceaccount my-service-account
# Ver Service Accounts kubectl get serviceaccounts
# Describir Service Account kubectl describe serviceaccount my-service-account
# Service Account declarativo apiVersion: v1 kind: ServiceAccount metadata:
name: build-robot namespace: default secrets: - name: build-robot-secret
imagePullSecrets: - name: my-registry-secret
```

5.2.2 Uso de Service Accounts en Pods

```
apiVersion: v1 kind: Pod metadata: name: my-pod spec: serviceAccountName:
my-service-account containers: - name: my-container image: nginx volumeMounts:
- name: sa-token mountPath: /var/run/secrets/kubernetes.io/serviceaccount
readOnly: true volumes: - name: sa-token projected: sources: -
serviceAccountToken: path: token expirationSeconds: 3607 - configMap: name:
kube-root-ca.crt items: - key: ca.crt path: ca.crt - downwardAPI: items: - path:
namespace fieldRef: fieldPath: metadata.namespace
```

5.3 Security Contexts

Los Security Contexts definen configuraciones de seguridad para pods y contenedores.

5.3.1 Security Context a nivel de Pod

P

5.3.2 Políticas de Seguridad Recomendadas

Tip Profesional

Mejores Prácticas de Seguridad:

- Usar usuarios no-root
- Deshabilitar escalation de privilegios
- Solo agregar capabilities necesarias
- Usar readOnlyRootFilesystem cuando sea posible
- Implementar seccomp profiles

5.4 Network Policies

Las Network Policies actúan como firewalls para pods, controlando el tráfico de red.

5.4.1 Componentes de Network Policy

podSelector: Selecciona pods a los que aplica la política

policyTypes: Ingress, Egress, o ambos

ingress: Reglas para tráfico entrante

egress: Reglas para tráfico saliente

5.4.2 Ejemplo de Network Policy Completa

```
apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name:
test-network-policy namespace: default spec: podSelector: matchLabels:
role: db policyTypes: - Ingress - Egress ingress: - from: - ipBlock: cidr:
172.17.0.0/16 except: - 172.17.1.0/24 - namespaceSelector: matchLabels: name:
web - podSelector: matchLabels: role: frontend ports: - protocol: TCP port: 6379
egress: - to: - ipBlock: cidr: 10.0.0.0/24 ports: - protocol: TCP port: 5978
```

5.4.3 Ejemplos Prácticos de Network Policies

r

5.5 Pod Security Standards

Los Pod Security Standards definen tres políticas para controlar aspectos de seguridad de pods.

5.5.1 Niveles de Pod Security

Privileged: Sin restricciones (inseguro)

Baseline: Previene escalaciones conocidas

Restricted: Fuertemente restrictivo (más seguro)

5.5.2 Implementación con Admission Controller

```
# Namespace con Pod Security Standards apiVersion: v1 kind: Namespace metadata:  
name: secure-namespace labels: pod-security.kubernetes.io/enforce: restricted  
pod-security.kubernetes.io/audit: restricted pod-security.kubernetes.io/warn:  
restricted
```

6 Troubleshooting & Monitoring

Información

Esta sección cubre el 30 % del examen CKA y es crucial para la administración diaria de clusters Kubernetes.

6.1 Troubleshooting de Clusters

6.1.1 Verificación del Estado del Cluster

```
# Estado general del cluster kubectl cluster-info kubectl get nodes -o wide
kubectl get pods --all-namespaces
# Verificar componentes del control plane kubectl get pods -n kube-system
# Estado de componentes kubectl get componentstatuses Deprecated pero útil
# Información detallada de nodos kubectl describe nodes
```

6.1.2 Troubleshooting de Nodos

Error Común

Problemas comunes: Nodos NotReady, recursos insuficientes, problemas de red, kubelet no funcional.

```
# Verificar estado de nodos kubectl get nodes kubectl describe node <node-name>
# Logs del kubelet sudo journalctl -u kubelet -f
# Verificar recursos del sistema top free -h df -h
# Estado de contenedores sudo crictl ps sudo crictl pods
# Reiniciar kubelet si es necesario sudo systemctl restart kubelet
```

6.1.3 Troubleshooting de Pods

```
# Estados de pods problemáticos kubectl get pods --field-selector=status.phase=Failed
kubectl get pods --field-selector=status.phase=Pending
# Información detallada kubectl describe pod <pod-name> kubectl logs <pod-name>-c
<container-name> kubectl logs <pod-name>--previous
# Debugging interactivo kubectl exec -it <pod-name>-- /bin/bash kubectl debug
<pod-name>-it --image=busybox
# Eventos del cluster kubectl get events --sort-by='.lastTimestamp'
```

6.1.4 Problemas Comunes y Soluciones

ImagePullBackOff

Verificar nombre de imagen y tag

Comprobar conectividad al registry

Revisar image pull secrets

CrashLoopBackOff

Revisar logs del contenedor

Verificar comando de inicio

Comprobar health checks

Pending

Recursos insuficientes

Node affinity no satisfecha

Taints y tolerations

6.2 Troubleshooting de Redes

6.2.1 Conectividad entre Pods

```
# Probar conectividad básica kubectl run test-pod --image=busybox:1.28 --rm -it -- sh
# Dentro del pod de prueba: # ping <pod-ip># nslookup <service-name># wget -qO- http://<service-name>:<port>
# Verificar reglas iptables sudo iptables -L -n -v
# Estado de interfaces de red ip addr show ip route show
```

6.2.2 Troubleshooting de Services

```
# Verificar endpoints del service kubectl get endpoints <service-name>kubectl describe service <service-name>
# Probar service desde dentro del cluster kubectl run debug-pod --image=nicolaka/netshoot --rm -it -- sh
# Verificar kube-proxy kubectl get pods -n kube-system | grep kube-proxy kubectl logs -n kube-system <kube-proxy-pod>
# Verificar reglas de proxy sudo iptables -t nat -L -n -v
```

6.3 Monitoring con Métricas

6.3.1 Metrics Server

```
# Instalar Metrics Server kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
# Verificar instalación kubectl get deployment metrics-server -n kube-system
# Ver métricas de nodos kubectl top nodes
# Ver métricas de pods kubectl top pods kubectl top pods --all-namespaces
```

6.3.2 Comandos de Monitoreo Útiles

```
# Uso de recursos por namespace kubectl top pods --sort-by=cpu -n <namespace>
kubectl top pods --sort-by=memory -n <namespace>
# Métricas en tiempo real watch kubectl top nodes watch kubectl top pods --all-namespaces
# Descripción de recursos con límites kubectl describe limitrange -n <namespace>
kubectl describe resourcequota -n <namespace>
```

6.3.3 Troubleshooting de Performance

Tip Profesional

Herramientas de Análisis:

- kubectl top para métricas básicas
- kubectl describe para eventos y estado
- Logs de aplicación para errores específicos
- Herramientas del sistema (htop, iostat, netstat)

6.4 Backup y Disaster Recovery

6.4.1 Estrategias de Backup

Backup de etcd (más crítico)

Backup de configuraciones (YAML manifests)

Backup de datos persistentes

Backup de certificados y secrets

6.4.2 Script de Backup Automatizado

```
#!/bin/bash # Automated backup script
DATE=$(date + $BACKUPDIR = "/backup/DATE"mkdir -p $BACKUPDIR
# etcd backup ETCDCTL_API=3 etcdctl snapshot save $BACKUPDIR/etcd --snapshot.db
--endpoints = https://127.0.0.1:2379
--cacert = /etc/kubernetes/pki/etcd/ca.crt
--cert = /etc/kubernetes/pki/etcd/server.crt
--key = /etc/kubernetes/pki/etcd/server.key
# Backup de manifests kubectl get all --all-namespaces -o yaml
>$BACKUPDIR/all --resources.yaml
# Backup de secrets kubectl get secrets --all-namespaces -o yaml
>$BACKUPDIR/all --secrets.yaml
# Backup de configmaps kubectl get configmaps --all-namespaces -o yaml
>$BACKUPDIR/all --configmaps.yaml
echo "Backup completed in $BACKUPDIR"
```

7 Comandos Esenciales y Cheat Sheet

7.1 Comandos Fundamentales

7.1.1 Gestión Básica

```
# Información del cluster kubectl cluster-info kubectl version kubectl config
view
# Gestión de contextos kubectl config get-contexts kubectl config
current-context kubectl config use-context <context-name>
# Namespaces kubectl get namespaces kubectl create namespace <name>kubectl
delete namespace <name>
```

7.1.2 Pods y Deployments

```
# Pods kubectl run <pod-name>--image=<image>--port=<port>kubectl get pods -o
wide kubectl describe pod <pod-name>kubectl logs <pod-name>-f kubectl exec -it
<pod-name>-- /bin/bash kubectl delete pod <pod-name>
# Deployments kubectl create deployment <name>--image=<image>kubectl get
deployments kubectl scale deployment <name>--replicas=<number>kubectl rollout
status deployment/<name>kubectl rollout history deployment/<name>kubectl rollout
undo deployment/<name>
```

7.1.3 Services y Networking

```
# Services kubectl expose deployment <name>--port=<port>--type=<type>kubectl get
services kubectl describe service <service-name>
# Ingress kubectl get ingress kubectl describe ingress <ingress-name>
# Network troubleshooting kubectl run netshoot --image=nicolaka/netshoot --rm
-it -- bash kubectl port-forward pod/<pod-name><local-port>:<pod-port>
```

7.2 Comandos de Troubleshooting

```
# Estado del cluster kubectl get nodes kubectl get pods --all-namespaces kubectl
get events --sort-by='.lastTimestamp'
# Debugging de pods kubectl describe pod <pod-name>kubectl logs
<pod-name>--previous kubectl exec -it <pod-name>-- sh
# Métricas kubectl top nodes kubectl top pods --all-namespaces
# Debugging de nodos kubectl describe node <node-name>kubectl get pods
--field-selector spec.nodeName=<node-name>
```

7.3 Comandos de Administración

```
# Drain y cordon de nodos kubectl drain <node-name>--ignore-daemonsets
--delete-emptydir-data kubectl cordon <node-name>kubectl uncordon <node-name>
# Etiquetas y annotations kubectl label nodes <node-name><key>=<value>kubectl
annotate pods <pod-name><key>=<value>
# Secrets y ConfigMaps kubectl create secret generic <name>--from-literal=<key>=<va
create configmap <name>--from-file=<file>
```

8 Ejemplos Prácticos del Examen CKA

Importante

Esta sección incluye escenarios prácticos similares a los del examen CKA real. Practica estos ejercicios para prepararte adecuadamente.

8.1 Escenarios de Cluster Architecture

8.1.1 Escenario 1: Backup y Restore de etcd

Problema: Realizar un backup de etcd y restaurarlo en caso de falla.

```
# 1. Realizar backup de etcd ETCDCCTL_API=3 etcdctl snapshot save
/opt/etcd-backup.db
--endpoints=https://127.0.0.1:2379
--cacert=/etc/kubernetes/pki/etcd/ca.crt
--cert=/etc/kubernetes/pki/etcd/server.crt
--key=/etc/kubernetes/pki/etcd/server.key
# 2. Verificar el backup ETCDCCTL_API=3 etcdctl snapshot status
/opt/etcd-backup.db
# 3. Restaurar (en caso necesario) sudo systemctl stop kubelet sudo systemctl
stop etcd
ETCDCCTL_API=3 etcdctl snapshot restore /opt/etcd-backup.db
--data-dir=/var/lib/etcd-restore
# 4. Actualizar manifest de etcd y reiniciar sudo vim
/etc/kubernetes/manifests/etcd.yaml sudo systemctl start kubelet
```

8.1.2 Escenario 2: Actualización de Cluster

Problema: Actualizar un cluster de Kubernetes de v1.28 a v1.29.

```
# 1. Verificar versión actual kubectl get nodes
# 2. Actualizar control plane sudo apt update sudo apt-mark unhold kubeadm sudo
apt install -y kubeadm=1.29.0-00 sudo apt-mark hold kubeadm
# 3. Planificar actualización sudo kubeadm upgrade plan
# 4. Aplicar actualización sudo kubeadm upgrade apply v1.29.0
# 5. Actualizar kubelet y kubectl kubectl drain control-plane
--ignore-daemonsets sudo apt-mark unhold kubelet kubectl sudo apt install
-y kubelet=1.29.0-00 kubectl=1.29.0-00 sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload sudo systemctl restart kubelet kubectl uncordon
control-plane
```

8.2 Escenarios de Networking

8.2.1 Escenario 3: Configurar Ingress

Problema: Exponer una aplicación usando Ingress con path-based routing.


```
# 1. Crear deployment kubectl create deployment web-app --image=nginx:1.20
kubectl expose deployment web-app --port=80 --target-port=80
# 2. Crear Ingress apiVersion: networking.k8s.io/v1 kind: Ingress metadata:
name: web-app-ingress annotations: nginx.ingress.kubernetes.io/rewrite-target:
/ spec: ingressClassName: nginx rules: - host: myapp.example.com http: paths: -
path: /app pathType: Prefix backend: service: name: web-app port: number: 80
# 3. Aplicar y verificar kubectl apply -f ingress.yaml kubectl get ingress
```

8.3 Escenarios de Security

8.3.1 Escenario 4: Configurar RBAC

Problema: Crear un usuario que solo pueda leer pods en el namespace "development".

8.4 Escenarios de Troubleshooting

8.4.1 Escenario 5: Pod no puede iniciar

Problema: Un pod está en estado "Pending" no puede programarse.

```
# 1. Verificar estado del pod kubectl get pods kubectl describe pod <pod-name>
# 2. Verificar eventos kubectl get events --sort-by='.lastTimestamp'
# 3. Verificar recursos de nodos kubectl top nodes kubectl describe nodes
# 4. Verificar taints y tolerations kubectl get nodes -o
custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
# 5. Soluciones comunes: # - Agregar resources requests # - Agregar tolerations
para taints # - Verificar nodeSelector/affinity
```

9 Apéndices

9.1 Recursos Útiles

9.1.1 Documentación Oficial

[Documentación de Kubernetes](#)

[Kubectl Cheat Sheet](#)

[Conceptos de Kubernetes](#)

[Tareas y Tutoriales](#)

9.1.2 Herramientas de Práctica

[killer.sh](#) - Simulador de examen CKA

[KodeKloud](#) - Laboratorios interactivos

[Kubernetes The Hard Way](#)

9.2 Configuración de Laboratorio

9.2.1 Ambiente con Vagrant

```
# Vagrantfile para laboratorio K8s Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/20.04"
  Master node config.vm.define "master" do |master|
    master.vm.hostname = "k8s-master"
    master.vm.network "private_network", ip: "192.168.56.10"
    master.vm.provider "virtualbox" do |v|
      v.memory = 2048
      v.cpus = 2
    end
  end
  Worker nodes (1..2).each do |i|
    config.vm.define "worker#{i}" do |worker|
      worker.vm.hostname = "k8s-worker#{i}"
      worker.vm.network "private_network", ip: "192.168.56.1#{i}"
      worker.vm.provider "virtualbox" do |v|
        v.memory = 1024
        v.cpus = 1
      end
    end
  end
end
```

9.2.2 Script de Instalación Automatizada

9.3 Glosario de Términos

API Server: Componente que expone la API de Kubernetes

CNI: Container Network Interface - especificación para plugins de red

CRI: Container Runtime Interface - especificación para runtimes

DaemonSet: Asegura que todos los nodos ejecuten una copia de un pod

etcd: Base de datos distribuida que almacena el estado del cluster

HPA: Horizontal Pod Autoscaler - escala pods automáticamente

RBAC: Role-Based Access Control - control de acceso basado en roles

Service Mesh: Capa de infraestructura para comunicación entre servicios

StatefulSet: Gestiona aplicaciones con estado

VPA: Vertical Pod Autoscaler - ajusta recursos de pods automáticamente

10 Conclusión

Información

Esta guía completa de Kubernetes CKA 2024 cubre todos los aspectos esenciales para la administración de clusters Kubernetes y la preparación para el examen CKA.

10.1 Puntos Clave para el Examen

Práctica Hands-on: El examen es 100 % práctico

Tiempo de Gestión: Tienes 2 horas para 15-20 tareas

Documentación: Puedes usar la documentación oficial

kubectl: Domina los comandos esenciales

Troubleshooting: Representa el 30 % del examen

10.2 Plan de Estudio Recomendado

Tip Profesional

Semana 1-2: Arquitectura y conceptos básicos

Semana 3-4: Instalación y configuración

Semana 5-6: Networking y storage

Semana 7-8: Security y RBAC

Semana 9-10: Troubleshooting y práctica intensiva

Semana 11-12: Simulacros de examen

10.3 Comandos para Memorizar

Los comandos más importantes que debes dominar:

```
kubectl get/describe/delete <resource>kubectl create/apply -f <file>kubectl  
logs/exec -it <pod>kubectl drain/cordon/uncordon <node>kubectl scale/rollout  
<deployment>kubectl auth can-i <verb><resource>etcdctl snapshot save/restore  
kubeadm init/join/upgrade
```

10.4 Última Recomendación

Importante

La práctica constante en un ambiente real es la clave del éxito. Instala tu propio cluster, experimenta con todos los comandos y escenarios de esta guía. ¡Mucha suerte en tu examen CKA!

¡Feliz aprendizaje de Kubernetes!

♥ Creado con amor para la comunidad de DevOps